VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

WEEK-2

Q. Simulate the following CPU scheduling algorithms a) Priority b) Round robin.

1.a) Priority scheduling algorithm preemptive case.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT), Burst Time (BT), and Priority (PR) for each process.
4. Assign process IDs (P1, P2, ..., Pn).
5. Initialize: Remaining Time (RT) = Burst Time (BT) for each process.
6. time = 0 and completed = 0 AND Mark all processes as not completed.
7. While completed < n: Find all processes that have: Arrived (AT ≤ time), and Remaining Time (RT) > 0.
8. Among these, select the process with the highest priority (smallest priority number).
9. If two processes have the same priority, choose the one with the earlier Arrival Time.
10. If no process has arrived, increment time by 1.
11. Execute the selected process for 1 unit of time.
12. Decrease its Remaining Time: RT = RT - 1
13. Increment the current time: time = time + 1
14. If the Remaining Time becomes 0: Mark the process as completed.
15. Calculate Completion Time (CT): CT = time
16. Calculate Turnaround Time (TAT): TAT = CT - AT
17. Calculate Waiting Time (WT): WT = TAT - BT
18. Increment the completed process count. AND Repeat Steps 6–10 until all processes are completed.
19. Calculate: Average Waiting Time = ΣWT / n and Average Turnaround Time = ΣTAT / n
20. Display the Process ID, Arrival Time, Burst Time, Priority, Completion Time, Waiting Time, and Turnaround Time.
21. Stop.

PROBLEM:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

PROGRAM:

```
#include <stdio.h>

int main() {
    int n, i, time = 0, completed = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], rt[n];
    int ct[n], tat[n], wt[n], done[n];

    printf("\nEnter Arrival Time, Burst Time and Priority for each process:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        rt[i] = bt[i];
        done[i] = 0;
    }

    while (completed < n) {
        int idx = -1;

        for (i = 0; i < n; i++) {
            if (at[i] <= time && done[i] == 0) {
                if (idx == -1 || pr[i] < pr[idx])
                    idx = i;
            }
        }

        if (idx == -1) {
            time++;
            continue;
        }

        rt[idx]--;
        time++;

        if (rt[idx] == 0) {
            done[idx] = 1;
            completed++;

            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
```

```
            wt[idx] = tat[idx] - bt[idx];
        }
    }

    float avgWT = 0, avgTAT = 0;

    printf("\n----------------------------------------------------------------\n");
    printf("Process\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    printf("----------------------------------------------------------------\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               i + 1, at[i], bt[i], pr[i],
               ct[i], tat[i], wt[i]);

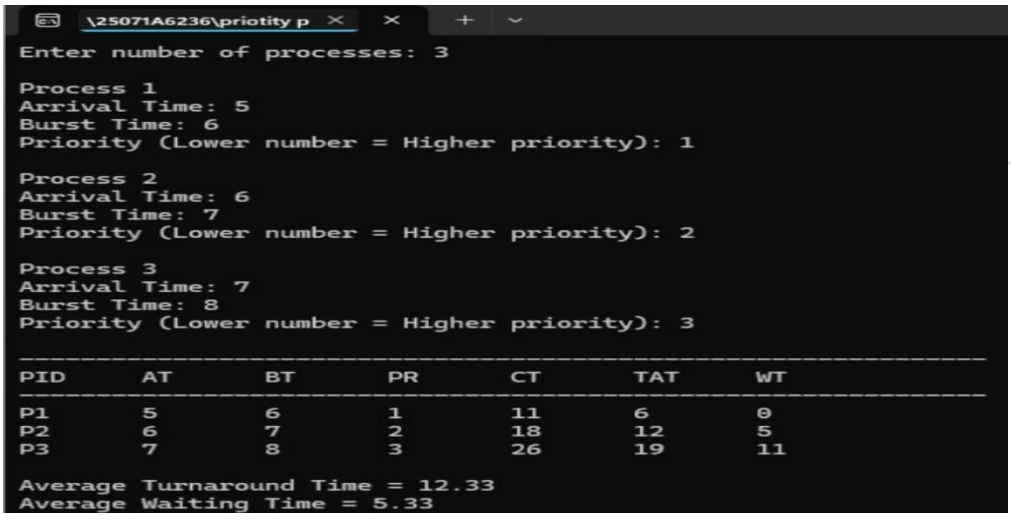
        avgWT += wt[i];
        avgTAT += tat[i];
    }

    printf("----------------------------------------------------------------\n");
    printf("Average Waiting Time = %.2f\n", avgWT / n);
    printf("Average Turnaround Time = %.2f\n", avgTAT / n);

    return 0;
}
```

OUTPUT:

TEST CASE 1:



TEST CASE 2:

```
p\25071A6236\priotity
Enter number of processes: 3

Process 1
Arrival Time: 2
Burst Time: 3
Priority (Lower number = Higher priority): 2

Process 2
Arrival Time: 4
Burst Time: 6
Priority (Lower number = Higher priority): 1

Process 3
Arrival Time: 9
Burst Time: 7
Priority (Lower number = Higher priority): 3

------------------------------------------------------------------
PID     AT      BT      PR      CT      TAT     WT
------------------------------------------------------------------
P1      2       3       2       11      9       6
P2      4       6       1       10      6       0
P3      9       7       3       18      9       2

Average Turnaround Time = 8.00
Average Waiting Time = 2.67
```

TEST CASE 3:

```
\25071A6236\priotity p
Enter number of processes: 3

Process 1
Arrival Time: 7
Burst Time: 6
Priority (Lower number = Higher priority): 2

Process 2
Arrival Time: 7
Burst Time: 5
Priority (Lower number = Higher priority): 3

Process 3
Arrival Time: 4
Burst Time: 6
Priority (Lower number = Higher priority): 1

------------------------------------------------------------------
PID     AT      BT      PR      CT      TAT     WT
------------------------------------------------------------------
P1      7       6       2       16      9       3
P2      7       5       3       21      14      9
P3      4       6       1       10      6       0

Average Turnaround Time = 9.67
Average Waiting Time = 4.00
```

2)a) Round Robin scheduling algorithm when time quantum is 2 and when time quantum is 10.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT) and Burst Time (BT) for each process.
4. Read the Time Quantum (TQ).
5. Assign process IDs (P1, P2, ..., Pn).
6. Initialize: Remaining Time (RT) = Burst Time (BT) for each process.
7. time = 0 and completed = 0
8. Mark all processes as not completed.
9. Add all processes that have arrived (AT ≤ time) to the Ready Queue.
10. While completed < n:
11. If the Ready Queue is empty:
    a. Increment time by 1.
    b. Add any newly arrived processes to the Ready Queue.
12. Otherwise:
    a. Remove the first process from the Ready Queue.
    b. If its Remaining Time (RT) is greater than the Time Quantum (TQ):
        i. Execute the process for TQ units.
        ii. RT = RT − TQ
        iii. time = time + TQ
    c. Else:
        i. Execute the process until completion.
        ii. time = time + RT
        iii. RT = 0
        iv. Calculate Completion Time (CT): CT = time
        v. Calculate Turnaround Time (TAT): TAT = CT − AT
        vi. Calculate Waiting Time (WT): WT = TAT − BT
        vii. Mark the process as completed.
        viii. Increment completed by 1.
    d. Add all newly arrived processes to the Ready Queue.
    e. If the current process is not completed, insert it at the rear of the Ready Queue.
13. Repeat Step 8 until all processes are completed.
14. Calculate: Average Waiting Time = ΣWT / n and Average Turnaround Time = ΣTAT / n
15. Display the Process ID, Arrival Time, Burst Time, Completion Time, Waiting Time, and Turnaround Time. Stop.

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

PROBLEM:

PROGRAM:

```
#include <stdio.h>
int main() {
    int n, i, tq = 2, time = 0, completed = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int bt[n], rt[n], ct[n], tat[n], wt[n];
    printf("Enter Burst Time for each process:\n");
    for(i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    while(completed < n) {
        for(i = 0; i < n; i++) {
            if(rt[i] > 0) {
                if(rt[i] <= tq) {
                    time += rt[i];
                    rt[i] = 0;
                    ct[i] = time;
                    completed++;
                } else {
                    rt[i] -= tq;
                    time += tq
```

```
    }
}
    }
    }
    float avgWT = 0, avgTAT = 0;
    printf("\n----------------------------------------\n");
    printf("Process\tBT\tCT\tTAT\tWT\n");
    printf("----------------------------------------\n");
    for(i = 0; i < n; i++) {
        tat[i] = ct[i];      // Arrival Time = 0
        wt[i] = tat[i] - bt[i];
        printf("P%d\t%d\t%d\t%d\t%d\n",
              i + 1, bt[i], ct[i], tat[i], wt[i]);
        avgWT += wt[i];
        avgTAT += tat[i];
    }
    printf("----------------------------------------\n");
    printf("Average Waiting Time = %.2f\n", avgWT / n);
    printf("Average Turnaround Time = %.2f\n", avgTAT / n);
    return 0;
}
```

OUTPUT:

TEST CASE 1:TQ=2

```
25071A6236_II\week_1(7)
Enter number of processes: 3
Enter Burst Time for each process:
P1: 12
P2: 21
P3: 7

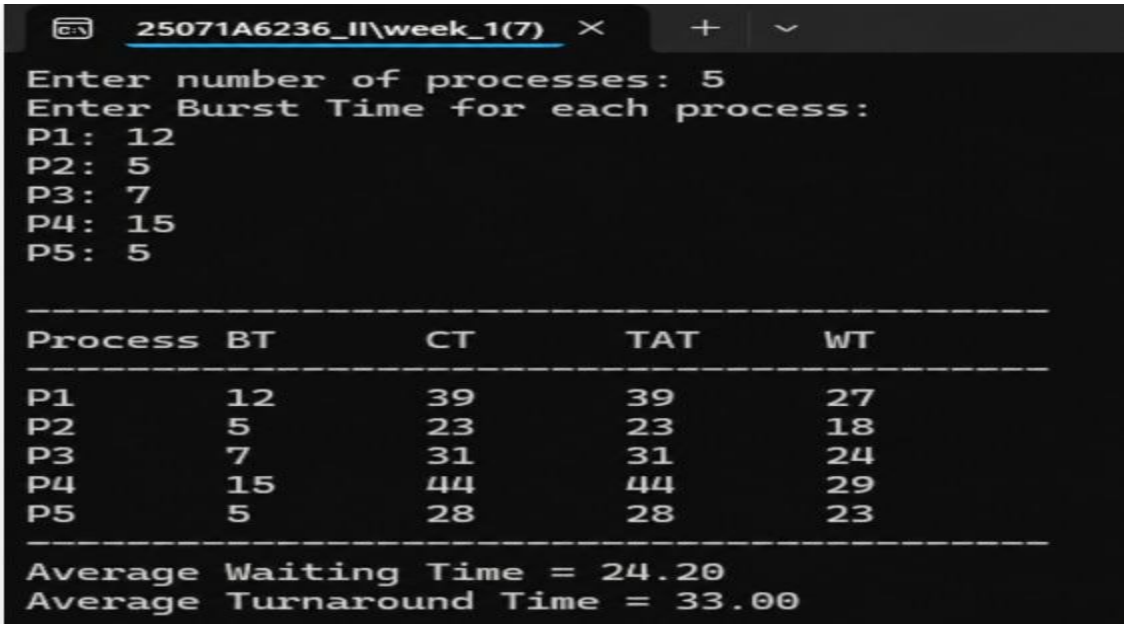
----------------------------------------
Process BT       CT       TAT      WT
----------------------------------------
P1      12       29       29       17
P2      21       40       40       19
P3      7        23       23       16
----------------------------------------
Average Waiting Time = 17.33
Average Turnaround Time = 30.67
```

TEST CASE 2: TQ=2

```
25071A6236_II\week_1(7)
Enter number of processes: 4
Enter Burst Time for each process:
P1: 12
P2: 3
P3: 9
P4: 22

----------------------------------------
Process BT       CT       TAT      WT
----------------------------------------
P1      12       34       34       22
P2      3        11       11       8
P3      9        30       30       21
P4      22       46       46       24
----------------------------------------
Average Waiting Time = 18.75
Average Turnaround Time = 30.25
```

TEST CASE 3: TQ=2



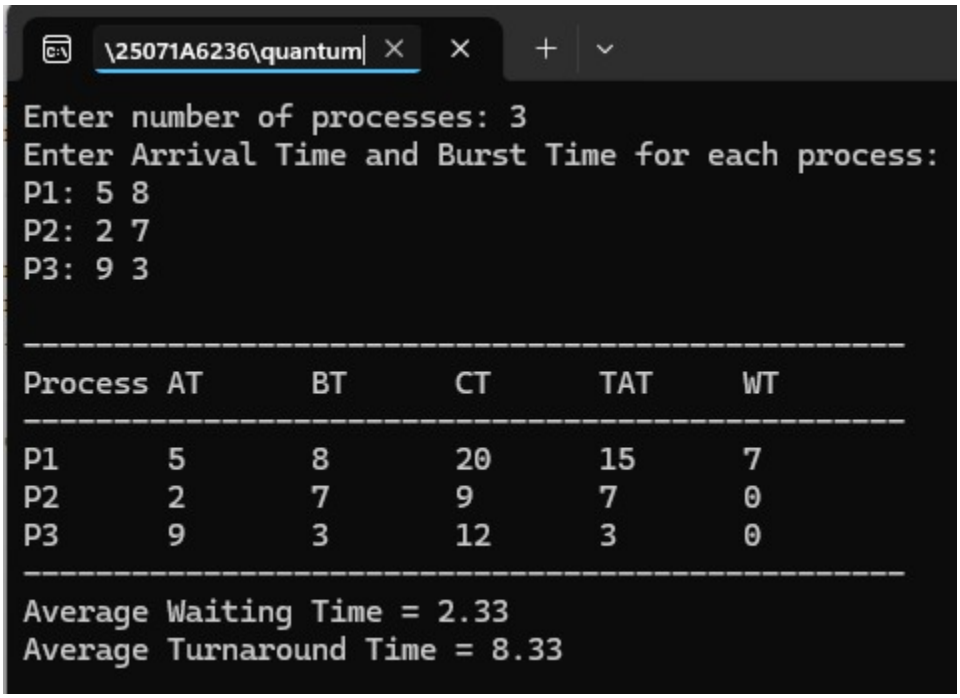
PROGRAM:

```
#include <stdio.h>
int main() {
    int n, tq = 10;
    int i, time = 0, completed = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int at[n], bt[n], rt[n];
    int ct[n], tat[n], wt[n];
    printf("Enter Arrival Time and Burst Time for each process:\n");
    for(i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }
    while(completed < n) {
        int found = 0;
        for(i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0) {
                found = 1;
                if(rt[i] <= tq) {
                    time += rt[i];
                    rt[i] = 0;
                    ct[i] = time;
```

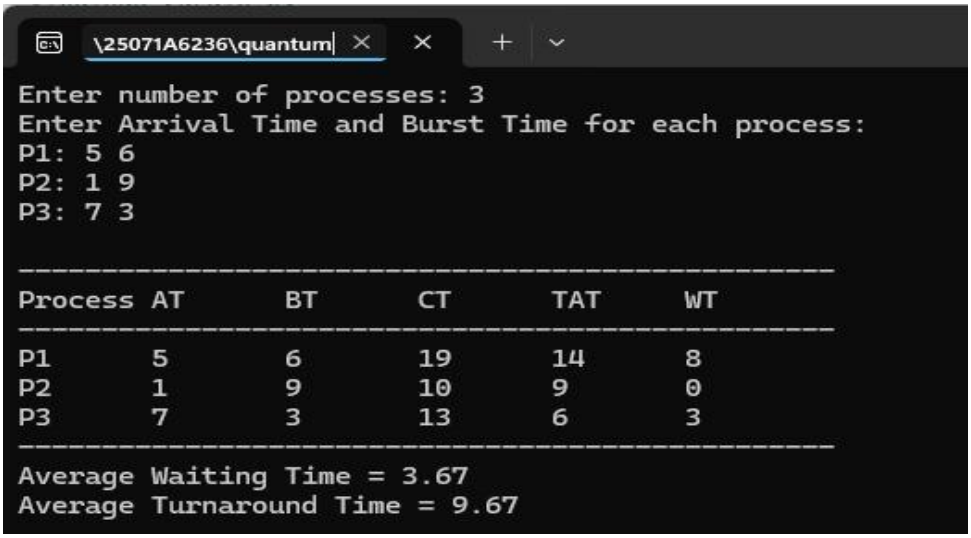
```
completed++;
                } else {
                  rt[i] -= tq;
                  time += tq;
                }
            } }
        if(!found)
            time++;
    }
    float avgWT = 0, avgTAT = 0;
    printf("\n----------------------------------------------\n");
    printf("Process\tAT\tBT\tCT\tTAT\tWT\n");
    printf("----------------------------------------------\n");
    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
              i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
        avgWT += wt[i];
        avgTAT += tat[i];
    printf("----------------------------------------------\n");
    printf("Average Waiting Time = %.2f\n", avgWT / n);
    printf("Average Turnaround Time = %.2f\n", avgTAT / n);
    return 0;   }
```

OUTPUT:

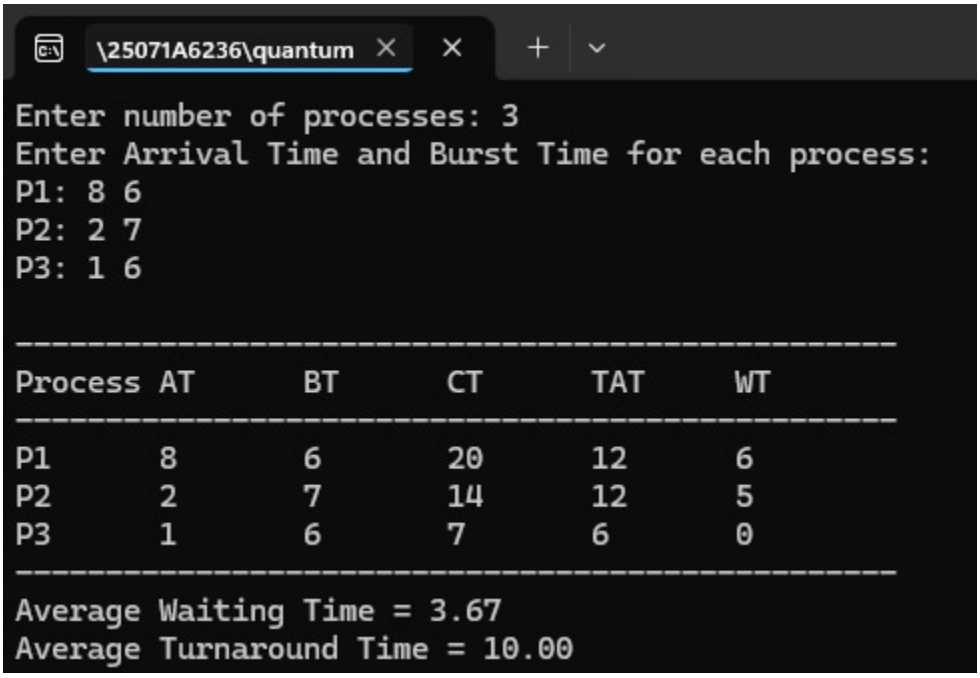
TEST CASE 1: TQ=10



TEST CASE 2: TQ=10



TEST CASE 3: TQ=10



VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Q) Banker's scheduling algorithm

ALGORITHM:

Algorithm: Banker's Algorithm for Deadlock Avoidance
Step 1: Start.
Step 2: Read the number of processes n and the number of resource types m.
Step 3: Read the **Allocation Matrix**, which represents the resources currently allocated to each process.
Step 4: Read the **Maximum Matrix**, which represents the maximum resources each process may need.
Step 5: Read the **Available Vector**, which represents the currently available resources.
Step 6: Calculate the **Need Matrix** using:
Need[i][j] = Max[i][j] - Allocation[i][j]
for every process i and resource type j.
Step 7: Initialize all processes as unfinished:
Finish[i] = 0
Also, initialize count = 0.
Step 8: Repeat the following steps until all processes are finished:

1. Set found = 0.
2. Search for an unfinished process i such that:
   Need[i][j] <= Available[j]
   for every resource type j.
3. If such a process is found:
   - Add its allocated resources back to the available resources:
     Available[j] = Available[j] + Allocation[i][j]
   - Add process i to the safe sequence.
   - Set Finish[i] = 1.
   - Increment count.
   - Set found = 1.
4. If no unfinished process can satisfy its remaining resource requirements (found == 0), the system is **not in a safe state**.

Step 9: If all processes are completed, the system is in a **safe state**.
Step 10: Display the safe sequence obtained.
Step 11: Stop.

PROBLEM:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

PROGRAM:

```
#include <stdio.h>

int main()
{
   int n, m;
   int allocation[10][10], max[10][10];
   int need[10][10], available[10];
   int finish[10] = {0};
   int safeSequence[10];

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);

   printf("\nEnter Allocation Matrix:\n");
   for (int i = 0; i < n; i++)
   {
      printf("P%d: ", i);
      for (int j = 0; j < m; j++)
      {
         scanf("%d", &allocation[i][j]);
      }
   }

   printf("\nEnter Maximum Matrix:\n");
   for (int i = 0; i < n; i++)
   {
      printf("P%d: ", i);
      for (int j = 0; j < m; j++)
      {
         scanf("%d", &max[i][j]);
      }
   }

   printf("\nEnter Available Resources:\n");
   for (int j = 0; j < m; j++)
   {
      scanf("%d", &available[j]);
   }
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < m; j++)
```

```
{
        need[i][j] = max[i][j] - allocation[i][j];
    }
 }
 printf("\nNeed Matrix:\n");
 for (int i = 0; i < n; i++)
 {
    printf("P%d: ", i);
    for (int j = 0; j < m; j++)
    {
       printf("%d ", need[i][j]);
    }
    printf("\n");
 }
 int count = 0;

 while (count < n)
 {
    int found = 0;

    for (int i = 0; i < n; i++)
    {
       if (finish[i] == 0)
       {
          int j;
          for (j = 0; j < m; j++)
          {
             if (need[i][j] > available[j])
             {
                break;
             }
          }
          if (j == m)
          {
             for (j = 0; j < m; j++)
             {
                available[j] += allocation[i][j];
             }

             safeSequence[count] = i;
             finish[i] = 1;
             count++;
             found = 1;
          }
       }
    }
    if (found == 0)
```

```
{
        printf("\nSystem is NOT in a safe state.\n");
        return 0;
    }
  }
  printf("\nSystem is in a SAFE state.\n");

  printf("Safe Sequence: ");

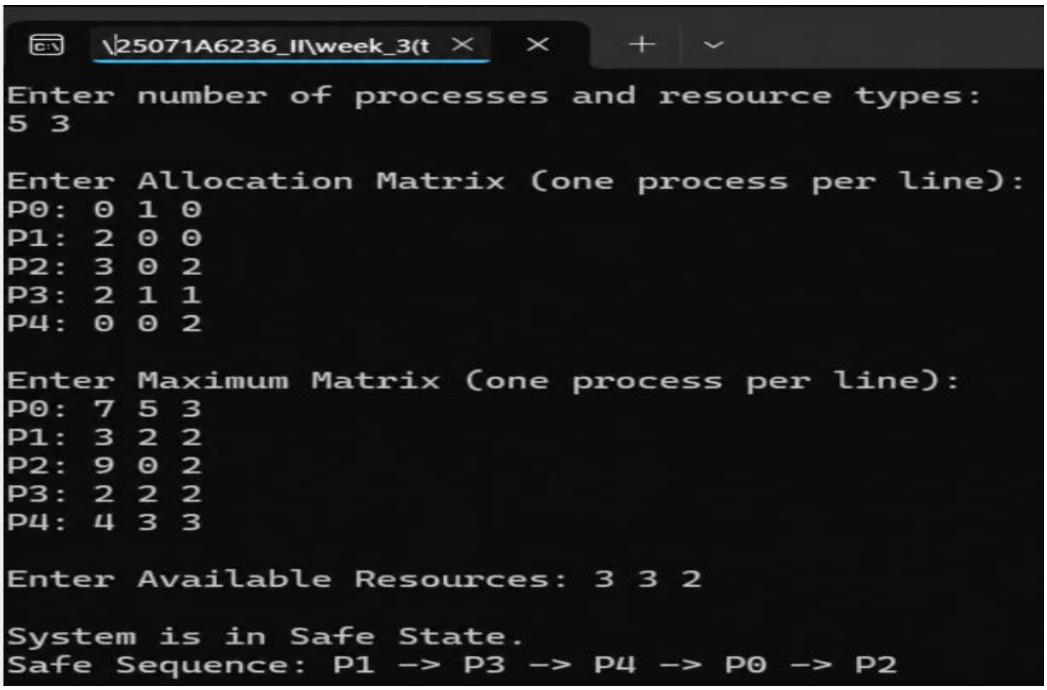
  for (int i = 0; i < n; i++)
  {
    printf("P%d", safeSequence[i]);

    if (i != n - 1)
      printf(" -> ");
  }

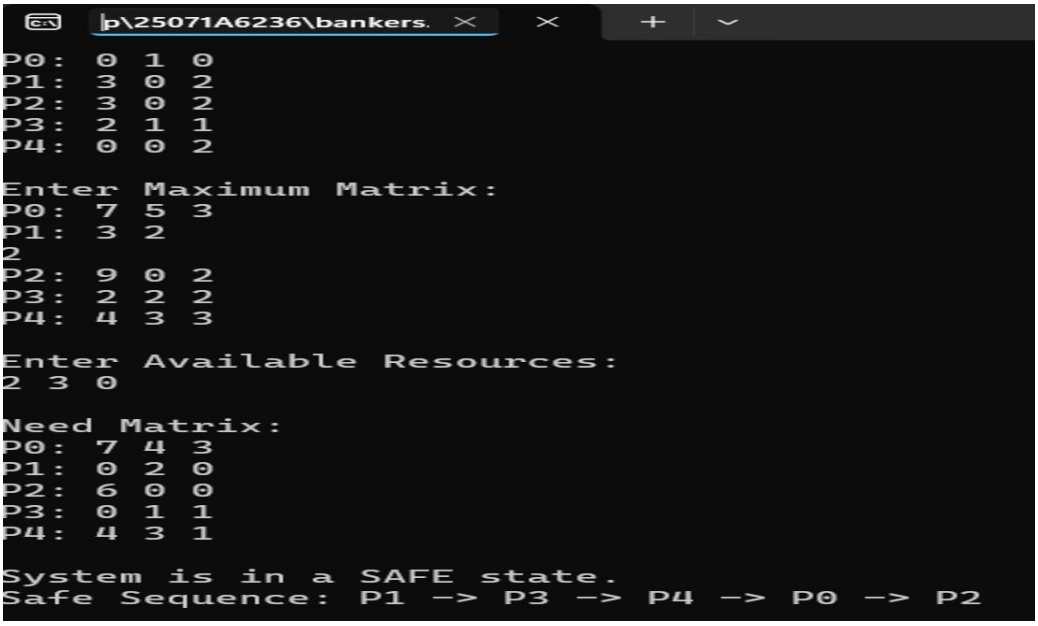
  printf("\n");

  return 0;
}
```

OUTPUT:
TEST CASE 1:



TEST CASE 2:



TEST CASE 3:

